

Difference Between `repartition()` and `coalesce()` in Apache Spark

Both `repartition()` and `coalesce()` are used to **change the number of partitions** in a DataFrame or RDD.

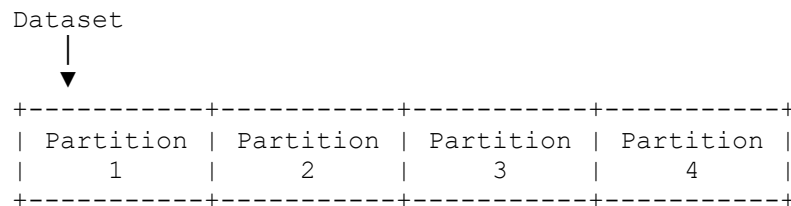
However, they work differently and are used in different situations.

What is a Partition?

A **partition** is a logical chunk of data.

Spark processes **one partition per task**.

Example:



4 Partitions → 4 Tasks

Changing the number of partitions affects **parallelism, performance, and resource utilization**.

What is `repartition()` ?

`repartition()` **redistributes data across all partitions** to create a new number of partitions.

It performs a **full shuffle**, meaning data is moved across the cluster.

Characteristics

- Can **increase** partitions.
- Can **decrease** partitions.
- Always performs a **shuffle**.

- Produces evenly balanced partitions.

Example

```
df2 = df.repartition(8)
```

If the original DataFrame has 4 partitions:

Before

P1 P2 P3 P4

```
repartition(8)
```

After

P1 P2 P3 P4 P5 P6 P7 P8

Spark redistributes the data so that each partition contains approximately the same amount of data.

What is `coalesce()` ?

`coalesce()` is primarily used to **reduce the number of partitions**.

Unlike `repartition()`, it tries to merge existing partitions **without a full shuffle**.

Characteristics

- Primarily decreases partitions.
- Avoids a full shuffle (when reducing partitions).
- Faster than `repartition()` for reducing partitions.
- May create uneven partition sizes.

Example

```
df2 = df.coalesce(2)
```

Suppose we have:

Before

P1 P2 P3 P4

After:

After

P1+P2 P3+P4

Spark merges partitions instead of redistributing all the data.

How They Work

repartition()

P1 P2 P3 P4

Shuffle

P1 P2 P3 P4 P5 P6

Every record may move to another partition.

coalesce()

P1 P2 P3 P4

Merge

P1+P2 P3+P4

Very little or no data movement is required.

Shuffle Behavior

Operation	Shuffle?
<code>repartition()</code>	✅ Yes (always)
<code>coalesce()</code>	❌ Usually no (when reducing partitions)

A shuffle is expensive because it transfers data across the network and writes intermediate data.

Performance

Since `coalesce()` avoids a full shuffle, it is generally **faster** when reducing the number of partitions.

`repartition()` is more expensive because Spark redistributes all the data.

Increasing Partitions

`repartition()`

Supports increasing partitions.

```
df.repartition(20)
```

Example:

```
4 Partitions
```

↓

```
20 Partitions
```

`coalesce()`

Although `coalesce()` technically accepts a larger number, it is **not intended** for increasing partitions because it does not redistribute data effectively.

```
df.coalesce(20)    # Not recommended
```

Use `repartition()` instead.

Typical Use Cases

Use `repartition()` When

- Increasing the number of partitions.
- You need evenly distributed data.
- Preparing data before a large join or aggregation.
- Avoiding data skew.

Example:

```
df = df.repartition(200)
```

Use `coalesce()` When

- Reducing partitions.
- Writing output files.
- Avoiding unnecessary shuffles.
- Improving performance when fewer partitions are sufficient.

Example:

```
df.coalesce(1).write.csv("output")
```

This writes a **single output file** (commonly used for demos or small datasets, but not recommended for very large datasets because it reduces parallelism).

Examples

Example 1: Increase Partitions

```
df = df.repartition(16)
```

Spark redistributes all data into 16 balanced partitions.

Example 2: Reduce Partitions

```
df = df.coalesce(4)
```

Spark merges partitions with minimal data movement.

Example 3: Write One Output File

```
df.coalesce(1).write.mode("overwrite").csv("output")
```

Spark reduces the partitions to one before writing, producing one data file (plus metadata files).

Comparison Table

Feature	<code>repartition()</code>	<code>coalesce()</code>
Purpose	Increase or decrease partitions	Primarily decrease partitions
Shuffle	✓ Always	✗ Usually no (when reducing)
Performance	Slower	Faster
Data Distribution	Evenly balanced	May be uneven
Increase Partitions	✓ Yes	⚠ Not recommended
Decrease Partitions	✓ Yes	✓ Yes
Typical Use	Rebalance data, increase parallelism	Reduce partitions before writing

Interview Questions

Q1. What is the difference between `repartition()` and `coalesce()`?

- `repartition()` changes the number of partitions by performing a **full shuffle**, resulting in evenly distributed data.
 - `coalesce()` primarily reduces the number of partitions by merging existing partitions, usually **without a full shuffle**, making it faster.
-

Q2. Which operation is faster?

`coalesce()` is generally faster when reducing partitions because it avoids a full shuffle.

Q3. Which operation can increase partitions?

`repartition()`.

Although `coalesce()` accepts a larger number, it is not designed for increasing partitions and is not recommended for that purpose.

Q4. Why is `repartition()` more expensive?

Because it performs a **full shuffle**, redistributing data across executors to create balanced partitions.

Q5. When should you use `coalesce(1)`?

Typically before writing a **small dataset** when you need a **single output file**. It should be avoided for very large datasets because it reduces parallelism and can become a performance bottleneck.

Interview Summary

- Both `repartition()` and `coalesce()` change the number of partitions in Spark.
- `repartition()` can increase or decrease partitions and **always performs a full shuffle**, resulting in balanced partitions.
- `coalesce()` is mainly used to **reduce** partitions and usually avoids a full shuffle, making it more efficient for that purpose.
- Use `repartition()` when you need better load balancing or more parallelism.
- Use `coalesce()` when reducing partitions, especially before writing output, to minimize unnecessary data movement.